Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Nombre Estudiantes	Cristian Tomala Cristian Valverde Juan Veintimmilla
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

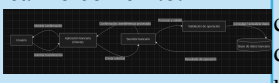
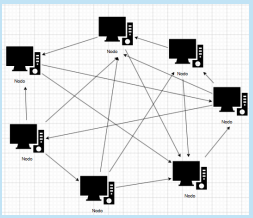
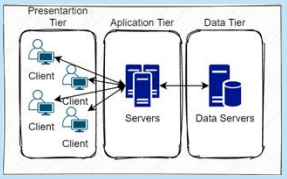
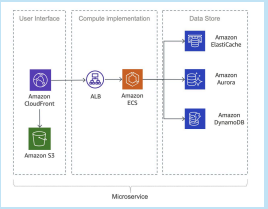
5. Procedimiento / Metodología

Fase 1 – Investigación (50 min). Dividimos de manera aleatoria los temas y proseguimos con la investigación de cada uno de ellos, realizando la consulta de papers y sitios web para realizar una comparación de conceptos; luego, completamos la matriz conformada por (estructura, ventajas, desventajas, escalabilidad, ejemplos). Una vez terminada la investigación, elaboramos nuestro ejemplo de arquitectura en draw.io.

Fase 2 – Socialización. Cada equipo expone 5 min. Realizamos (una sola matriz grupal) con base en los conceptos de cada grupo, previamente haciendo una comparativa con los conceptos investigados por nosotros.

Fase 3 – Reporte y entrega. Una vez finalizadas las exposiciones, redactamos independientemente las conclusiones; finalmente, realizamos una revisión de todo el documento realizado, analizando posibles errores o falantes para la entrega en la plataforma indicada.

Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
Estructura	Tiene una arquitectura centralizada porque los recursos y servicios están administrados por el servidor. Los clientes pueden realizar una o varias solicitudes a uno o varios servidores. [6]	Esta es una red descentralizada en la cual cada nodo actúa tanto como cliente como servidor y se divide en cuatro subtipos: Centralizado,Descentralizado ,estructurado y no estructurado. [1],[6]	Se divide por medio de capas separando las responsabilidades y dependencias específicas siendo sus interacciones únicamente por medio de una capa inferior. [5]	Se basa en una colección de componentes/servicios pequeños y autónomos, implementando una funcionalidad de negocio individual teniendo en cuenta el contexto delimitado. [4],[8]
Ventajas	- Facilita el control de usuarios y la seguridad, así como el mantenimiento y la actualización del servidor - Permite que varios clientes puedan acceder simultáneamente a los servicios logrando una administración centralizada de los recursos y datos	Tiene alta escalabilidad. Seguridad, si falla un ordenador el impacto es mínimo. Flexibilidad en el reparto de tareas.	Útil para plataformas en la nube Arquitectura fácil de manejar Puede darse uso en entornos variados Permite ver una clara separación de responsabilidades	Arquitectura ágil para la implementación y administración de nodos Permite la combinación de tecnologías Al ser nodos independientes, se aíslan los errores.
Desventajas	- Pueden ocurrir fallos cuando el número de clientes sea mayor al número que soporte el servidor	Requiere de mucho esfuerzo para administrarse. Rendimiento variable al depender de los nodos. La consistencia de los datos	Se evidencia un nivel medio lo cual genera latencia y complejidad sin aportar un valor significativo	Arquitectura compleja al presentar varios elementos a comparación de una arquitectura monolito. Presenta congestión por

	simultáneamente, esta dependencia del servidor llega afectar enormemente a la disponibilidad del servidor.	es más compleja y puede volverse restringida si estos se borran y no hay nadie más que tenga los datos.	Al ser monolito impide implementaciones independientes Las constantes solicitudes de usuario genera dificultades	varios servicios, generando latencia Menos facilidad en el control de versiones
Escalabilidad	Cuenta con una escalabilidad moderna, pues se puede aumentar la capacidad del servidor mediante escalamiento vertical.	Mientras haya más ordenadores perteneciendo a la red el rendimiento puede llegar a ser mayor.	Limitada, al ser casi idéntica a una arquitectura monolítica, siendo incapaz de aislar componentes específicos	Ilimitada, al basarse en pequeños nodos independientes, se pueden manejar de formas específicas sin el desperdicio de recursos por el uso de componentes adicionales
Ejemplos reales	Banco de Loja: el cliente solicita una transferencia mediante la aplicación móvil o web y el servidor procesa la operación y actualiza la información de la cuenta si la transferencia se realizó con éxito. 	BitTorrent este es un protocolo el cual se centra en el intercambio de datos de manera descentralizada, ayuda a poder descargar un archivo desde varios nodos al mismo tiempo los cuales tienen el mismo archivo. Tauri wow es un launcher el cual permite la descarga de diferentes expansiones de un videojuego. 	Sistema de Ventas Online, mediante un frontend que interactúa con un backend centralizado por medio frameworks junto a una base de datos. 	Amazon, el cual es capaz de separar las búsquedas, las gestiones de carritos, su almacenamiento de datos y reseñas de los usuarios, entre otros. 

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)
2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
	Estructura	Compuesto por cliente (aplicación) y el servidor que se encarga de recibir la solicitud del cliente	Se basa en la interconexión de distintos nodos (peers) en donde cada uno puede actuar como cliente y servidor	Organizada por medio de 3 o mas capas siendo la primera capa de presentación, la segunda capa presenta el negocio, finalmente la última capa se refiere a la capa de trazabilidad, finalmente está la capa de base de datos	La aplicación se divide en servicios pequeños los cuales trabajan juntos y cada servicio presenta una función concreta de negocio. Estos servicios presentan una única responsabilidad y son capaces de ayudarse entre sí.

GRUPO 1 ...	Ventajas	Organización entre las funciones de cliente-servidor Centraliza la información del software	Escalabilidad alta Mantiene la privacidad Propiedad compartida	organización jerárquica cada capa se encarga de una acción específica Da seguridad	Presenta la capacidad de alterar el codigo de cierta función sin la necesidad de interrumpir a las otras funciones.
	Desventajas	Saturación del sistema por la carga de peticiones	Alta volatilidad de nodos	Dificulta la coordinación entre todos los nodos, ya que tiene que atravesar demasiadas capas, lo que provocaría latencia.	Servicios separados puede generar latencia Mantenimiento más tardado por la identificación de errores en diferentes servicios
	Escalabilidad	Es escalable por el aumento de recursos	Escalabilidad alta y prácticamente global.	Presenta factores para promover la escalabilidad	Puede aumentar la capacidad de servicios, presentando una alta escalabilidad para aprovechar el hardware.
	Ejemplos reales	Sistema académico centrado en procesos de matriculación	Skype y BitTorrent	sistema bancario ya que contiene información sensible	SoundCLoud, ya que tenía dificultades de escalabilidad y mantenimiento
GRUPO 2 ...	Estructura	aplicativo dividido por dos componentes, cliente y servidor.	Nodos se conectan entre sí y comparten modelos, compuestos por cuatro modelos: Estructurada, Híbrida, No estructurada y Descentralizada/Pura.	Manejada en capas de manera horizontal presentando el principio de separación de responsabilidad	Arquitectura basada en el desglose de componentes independientes desplegadas de distintos servicios y se comunican por medio de apis, sus partes fundamentales son el servicio, el gateway y el cliente.
	Ventajas	Servidor centralizado donde todos los clientes pueden acceder a la misma información	No depende de un solo servidor, presentando una tolerancia a fallos.	Presenta aislamiento entre capas y la capacidad de generar pruebas entre capas	Libertad de elección de diferentes tecnologías para cada servicio.
	Desventajas	En caso de fallar el servidor todos los usuarios se ven afectados incluyendo a la empresa.	Como hay demasiados nodos, si existe fallo hay gran dificultad en encontrar la falla.	Presenta el riesgo de bloqueo tecnológico	El mantenimiento, al contar con demasiadas tecnologías, es más difícil atender a todas. Coherencia o consistencia de datos Costes de mantenimiento o despliegue de cada tecnología
	Escalabilidad	Alta escalabilidad,	Alta escalabilidad,	Tiene un nivel intermedio, su	Tiene un nivel alto debido a la

		aumentando la capacidad y ser independiente en el aspecto del apartado de clientes.	entre mas nodos mas fuerte es la red.	fortaleza es la separación de responsabilidad es de cada capa	separación de servicios del software, generando las actualización en servicios específicos.
	Ejemplos reales	Netflix, YouTube y tiendas online donde el cliente solicita y el servidor provee.	Criptomonedas, están en todo el mundo y se comunican constantemente entre sí.	health watcher aplicación web de gestión de salud en la cual los usuarios registrar tanto quejas como consultas de salud	JP Morgan usa microservicios en la nube para la flexibilidad que necesita un sistema financiero y no depender de un único proveedor para mantener un servicio confiable en todo momento.
GRUPO 3 ...	Estructura	Presenta 3 partes, el apartado del cliente, el servidor y la red.	Se tratan de diferentes nodos que actúan tanto como cliente como servidor	Se maneja mediante capas siendo la mas común por medio de 3 capas	Presenta API gateway
	Ventajas	Centraliza la información, lo que permite un fácil mantenimiento.	Compartir información, gracias a la red superpuesta en internet.	Cada capa se maneja de manera individual e independiente.	Cada microservicio puede ser mejorado sin afectar a los demás; además, es mucho más sencillo de manejar.
	Desventajas	Si el servidor falla todos los clientes pierden los servicios Si recibe muchas solicitudes puede caer	Puede presentar ataques e inestabilidad	En sistemas pequeños sería innecesaria la incorporación de demasiadas capas, lo que podría presentar latencia al pasar por todas estas capas.	Puede presentar inconsistencia, generando bucles y latencia entre los datos, al igual que presenta un costo masivo al querer mejorar el software
	Escalabilidad	presenta dos tipos, la vertical y la horizontal dependiendo el caso que corresponda	Mientras más nodos haya puede seguir creciendo de manera indefinida	alta debido a su distribución de capas	Tiene dos formas, horizontal, donde se pueden agregar muchos microservicios, y la vertical, donde, si uno falla, no va a afectar a los otros.
	Ejemplos reales	La estructura de un banco por medio del teléfono, la red y los datos que el banco presenta.	Base de datos como Gnutella para la descarga y compartir información	Un cooperativa manejado por capas	Uber con 3 servicios
GRUPO 4 ...	Estructura	Se basa en un cliente que realiza una solicitud y un servidor que responde a la misma	Una red de dispositivos manejando como cliente o servidor	Sistema dividido en funcionalidades diferentes,su estructura esta dividida en tres capas, presentación,	Estructura basada en por segmentos: Fronted, Api gateway, microservicios independientes, base de datos.

				media y almacenamiento.	
	Ventajas	Desacoplado, conecta múltiples clientes y maneja datos de uso frecuente por medio del servidor.	Descentralizado comunicación directa entre los nodos además de alta tolerancia a fallas por distribución de recursos	Estricta división de tareas cambios en la lógica interna por capas sin afectar las demás	Independencia total al actualizar, testear y desplegar sin afectar al resto tambien tiene libertad de lenguaje para cada módulo
	Desventajas	muchos clientes al mismo tiempo pueden provocar una caída depende de la disponibilidad del servidor	alta complejidad del software Presenta problemas de saturación a la banda ancha	una falla en un componente compartido arriesga todo el conjunto de la aplicación	Presenta un mayor costo global por la defensa de módulos filtración de datos por falta de seguridad
	Escalabilidad		Alta por medio de la unión de más nodos como la configuración de cada uno	Alta debido a que se pueden generar cambios como implementar nuevas funciones.	Alta y granular limitada por el presupuesto
	Ejemplos reales	Un sistema universitario de Cuenca.	bitcoins como también bitTorrent	Sistema de matrículas como sistemas de bancos	Billeteras digitales.
GRUPO 5 ...	Estructura	Cuenta con un cliente y servidor en donde se hacen peticiones.	cada nodo participante opera de manera descentralizada y funcionan como clientes y servidores	cuenta con tres capas de presentacion, negocios y datos	va desde una aplicacion dividida en pequeños servicios independientes
	Ventajas	Mayor seguridad y manejo de la información de los datos.	es tolerante a fallos al no tener un servidor central	muy utilizada y versatil facilmente escalable	el despliegue y la actualizacion es independiente y tolera fallos
	Desventajas	Si el servidor falla, presenta retraso.	su administracion y su coordinacion es mas compleja por la cantidad de nodos	perdida de rendimiento y latencia por el viaje de los datos entre las capas	se necesita una alta complejidad operativa
	Escalabilidad	Escalabilidad intermedia, cuenta con la capacidad de aumentar recursos.	Muy escalable cada nodo que ingresa para consumir recursos aportar	cada capa puede escalar de manera independiente	es altamente granular y eficaz para escalar de forma independiente

			tambien sus recursos		
	Ejemplos reales	IDK	BitTorrent, windows delivery optimization	aplicacion web de un carrito de compras en sitios de comercio y un sistema de banca en linea	uber y aplicaciones como spotify
GRUPO 6 ...	Estructura	Está centralizado en los clientes que realizan solicitudes hacia un servidor central el cual les responde	Nodos que actúan como clientes y servidores	tiene una capa de presentación, negocios y datos	cuenta con servicios pequeños que están desacoplados comunicados
	Ventajas	Tiene una arquitectura centralizada	alta escalabilidad	cada capa se centra en realizar su tarea en especifico	cuenta con una arquitectura ágil
	Desventajas	Puede dejar sin servicios si llega a colapsar el servidor	fallos en la seguridad	tiene una alta dependencia	arquitectura compleja para poder presentar varios elementos
	Escalabilidad	Necesita de aumento de hardware o de servidores para aligerar la carga	Mientras mas nodos llegue a tener hay mas recursos en la red	es escalable por las capas independientes	Esta se encuentra limitada al basarse en diferentes nodos
	Ejemplos reales	Correo electrónico como Gmail	Descarga de peliculas	sistemas bancarios	plataformas web como amazon

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Preguntas de Control:

¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?

El sistema distribuido se basa en la coordinación de múltiples computadoras independientes que actúan como una sola, con alta disponibilidad y tolerable a fallos, en cambio, el sistema centralizado basa todos sus procesos en un único servidor, creando un único punto de falla, limitando la escalabilidad de la misma [2].

Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.

La principal diferencia está en que la comunicación cliente-servidor, los usuarios clientes piden la información a un nodo central (usualmente el servidor), mientras que en P2P no existe nodo central, sino que los usuarios o dispositivos actúan tanto como clientes como servidores [6].

¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?

El middleware se trata de un puente entre softwares, conectando SO, bases de datos y aplicaciones para poder comunicarse, se ubica en la capa intermedia entre el SO subyacente y las aplicaciones del usuario [7].

En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?

La arquitectura de tres capas (de la arquitectura multicapas) divide la presentación (frontend), la lógica (backend) y los datos (base de datos), esto con el propósito de permitir una escalabilidad en una capa específica sin la necesidad de afectar a las demás [5].

¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?

La ventaja fundamental es el aislamiento de fallos entre servicios como la escalabilidad enfocándose en los servicios con alta demanda que presente el software, su principal desafío es la complejidad que requiere su

funcionamiento, al tener que gestionar las comunicaciones entre varios servicios hacia el usuario y la base de datos [3],[8].

¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?

La arquitectura estrella es P2P debido a lo ideal que distribuye la carga de archivos entre usuarios, siendo que al compartir archivos masivos, comparte fragmentos del mismo archivo con la demás gente, evitando el colapso del servidor y la fluidez de traspaso de información en la red [1].

¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?

influye en el apartado a mejorar, mientra que la escalabilidad horizontal añade más servidores y nodos, la escalabilidad vertical aumenta los recursos de un único servidor, es por ello que se elige en el contexto de la arquitectura distribuida, el uso del escalado horizontal, siendo más flexible y sostenible a largo plazo [2],[4].

¿Qué significa transparencia en un sistema distribuido y menciona dos tipos de transparencia?

la transparencia es la capacidad del sistema para disimular la complejidad, generando la idea de que es un trabajo de un solo equipo local, entre los dos tipos más comunes tenemos la transparencia de ubicación (el usuario no conoce la ubicación del servidor) y la transparencia de fallos (un nodo/servidor colapsa y el sistema lo recupera sin que el usuario lo note) [2].

Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?

La arquitectura vencedora es la P2P, debido a que los nodos presentes cuentan con autonomía, generando que no detenga la red y el traspaso de datos [1],[6].

En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?

El interés de conocer cómo podría funcionar a futuro nuestro software, por ello se tomó en cuenta la arquitectura de microservicios, separando responsabilidades y generando flexibilidad.

8. Evaluación

No aplica en esta practica

9. Bibliografía

[1] ReactiveProgramming.io, "Estilos arquitectónicos: Peer-to-Peer (P2P)", *ReactiveProgramming*. [En línea]. Disponible en: https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/p2p. [Accedido: 02-oct-2026].

[2] Amazon Web Services (AWS), "¿Qué es la computación distribuida?", *Amazon Web Services*. [En línea]. Disponible en: https://aws.amazon.com/es/what-is/distributed-computing/. [Accedido: 02-oct-2026].

[3] Amazon Web Services (AWS), "¿Qué son los microservicios?", *Amazon Web Services*. [En línea]. Disponible en: https://aws.amazon.com/es/microservices/. [Accedido: 02-oct-2026].

[4] Microsoft Azure, "Estilo de arquitectura de microservicios", *Microsoft Learn*. [En línea]. Disponible en:

https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices. [Accedido: 02-oct-2026].

[5] Microsoft Azure, "Estilo de arquitectura de N niveles", *Microsoft Learn*. [En línea]. Disponible en: https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/n-tier. [Accedido: 02-oct-2026].

[6] GeeksforGeeks, "Difference between Client-Server and Peer-to-Peer Network", *GeeksforGeeks*. [En línea]. Disponible en: https://www.geeksforgeeks.org/computer-networks/difference-between-client-server-and-peer-to-peer-network/. [Accedido: 02-oct-2026].

[7] Red Hat, "¿Qué es el middleware?", *Red Hat*. [En línea]. Disponible en: https://www.redhat.com/es/topics/middleware/what-is-middleware. [Accedido: 02-oct-2026].

[8] Red Hat, "¿Qué son los microservicios?", *Red Hat*. [En línea]. Disponible en: https://www.redhat.com/es/topics/microservices/what-are-microservices. [Accedido: 02-oct-2026].

[9] F. García de Zúñiga, "Todo sobre la arquitectura cliente-servidor," *Blog de Arsys*, 17-jul-2025. [En línea]. Disponible en: https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor .

[10] W. Li, W. Zhou, K. Yu, C. Feng, H. Wang, and D. Zhao, "R4: A Client Server Architecture Oriented Open Three Dimensional Reconstruction Layered Reference Model for Medical Images," in *Proc. 3rd Int. Symp. Intelligent Computing and Applications (ISICDM)*, Xi'an, China, Aug. 2019, pp. 48–54, doi: 10.1145/3364836.3364846.

[11] A. M. Khan, I. Arsov, M. Preda, S. Chabridon, and A. Beugnard, "Adaptable Client-Server Architecture for Mobile Multiplayer Games," in *Proc. SIMUTools 2010*, Torremolinos, Spain, 2010.

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente con FirmaEC. Firmado electrónicamente por: JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente con FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC